

DB 엔지니어 포트폴리오 - 실무 케이스 스터디

공공기관(군) 학사시스템을 약 3년 10개월간 유지보수하며 Tibero DB 운영, SQL 성능 개선, 데이터 변경·복구 작업을 담당했습니다.

항목	내용
기간	약 3년 10개월
업무	군 학사시스템 DB 유지보수
DBMS	Tibero
개발 환경	WebSquare, 전자정부프레임워크, Jeus
주요 역할	쿼리 성능 개선, 테이블 구조 변경, 데이터 수정·복구, 화면 구현·수정

※ 보안을 위해 실제 테이블명과 데이터는 일반화된 명칭으로 구성했습니다.

📄 전체 문서 PDF: [portfolio.pdf](#)

핵심 성과

- 다중 중첩 서브쿼리 구조 개선으로 종합성적 조회 시간 단축
- 운영 데이터 손실 없이 기능 추가에 따른 테이블 구조 변경 및 재구축 수행
- 조건 누락으로 잘못 변경된 데이터를 백업본과 Flashback Query로 복구, 이후 사전 검증·이중 확인 절차 강화
- 데이터 장애관련 문제 원인 분석과 테이블 전수 추적으로 찾아 정정하고 해결

목차

- [종합성적 조회 쿼리 구조 개선](#) — 조회 시간 10초 > 3초
- [기능 추가에 따른 테이블 구조 변경 및 재구축](#)
- [데이터 변경 작업 절차 수립 및 오삭제 복구](#)
- [데이터 장애 원인 추적 및 정정](#)

사례 1. 종합성적 조회 쿼리 구조 개선

다중 중첩 서브쿼리를 간소화해 종합성적 조회 시간을 10초에서 3초로 약 70% 단축했습니다.

문제 종합성적 조회 SQL이 3~5중으로 중첩된 서브쿼리로 구성되어 조회 응답이 약 10초 지연되었습니다. 구조가 복잡해 요구사항 변경 시 수정도 어려웠습니다.

분석 쿼리를 단계별로 분해해 각 서브쿼리의 역할을 확인했습니다. 동일 테이블을 여러 단계에서 반복 조회하거나 결과에 영향을 주지 않는 불필요한 중첩이 다수 있었습니다.

조치

- 결과에 영향이 없는 불필요한 중첩 단계 제거
- 반복 조회되던 서브쿼리를 통합해 쿼리 구조 간소화
- 수정 전후 조회 결과를 비교해 데이터 정합성 검증 후 반영

결과

항목	개선 전	개선 후
조회 시간	약 10초	약 3초
서브쿼리 중첩	3~5중	간소화

쿼리 가독성이 개선되어 이후 유지보수 작업도 쉬워졌습니다.

예시 쿼리 (가상, 일반화된 테이블명)

학생 1명의 연도별 종합성적을 4개 영역 점수와 서열로 산출하는 조회 쿼리입니다.

개선 전: 영역별 인라인뷰가 각각 기수 전체를 집계하고, 같은 서브쿼리가 반복되는 구조입니다. (sql/case1_before.sql)

```
-- 사례 1. 종합성적 조회 쿼리 - 개선 전 (가상, 일반화된 테이블명)
-- 영역별 인라인뷰가 각각 기수 전체를 집계하고, 같은 서브쿼리가 반복됨
SELECT :yy AS YY
, M.STD_NO, M.STD_NM
, TO_CHAR(M.STUDY_SCR, 'FM990.00') AS STUDY_SCR
, TO_CHAR(P.PRAC_SCR, 'FM990.00') AS PRAC_SCR
, TO_CHAR(F.FIT_SCR, 'FM990.00') AS FIT_SCR
, TO_CHAR(L.LIFE_SCR, 'FM990.00') AS LIFE_SCR
, TO_CHAR(M.STUDY_SCR*0.5 + P.PRAC_SCR*0.25
+ F.FIT_SCR*0.15 + L.LIFE_SCR*0.1, 'FM990.00') AS TOT_SCR
, TO_CHAR(K.TOT_RANK, 'FM990.0') AS TOT_RANK
FROM
-- 학업 점수
(SELECT A.STD_NO, A.STD_NM, B.AVG_POINT AS STUDY_SCR
FROM (SELECT S.STD_NO, S.STD_NM
, FN_CODE_NM('DEPT', S.DEPT_CD) AS DEPT_NM -- 미사용
, FN_CODE_NM('STAT', S.STAT_CD) AS STAT_NM -- 미사용
FROM STD_MASTER S, DEPT_INFO D
WHERE S.DEPT_CD = D.DEPT_CD(+)) A,
(SELECT R.STD_NO
, ROUND(SUM(R.CREDIT*R.POINT)/SUM(R.CREDIT), 2) AS AVG_POINT
, RANK() OVER (ORDER BY SUM(R.CREDIT*R.POINT)/SUM(R.CREDIT) DESC) AS RNK -- 미사용
FROM SUBJ_RESULT R
WHERE R.YY = :yy
AND R.SUBJ_DIV IN (SELECT CD FROM COMMON_CODE
WHERE GRP_CD = 'SUBJ_DIV' AND ATTR1 = 'G')
AND R.STD_NO IN (SELECT S2.STD_NO FROM STD_MASTER S2
WHERE S2.GRDT_NO = (SELECT GRDT_NO FROM STD_MASTER
WHERE STD_NO = :std_no))
GROUP BY R.STD_NO) B -- 기수 전체 집계
WHERE A.STD_NO = B.STD_NO
AND A.STD_NO = :std_no -- 마지막에 1명만 선택
ORDER BY B.AVG_POINT DESC) M,
-- 실습 점수
(SELECT C.STD_NO
, ROUND(C.T1*0.2 + C.T2*0.2 + C.SUMMER*0.6, 2) AS PRAC_SCR
, (SELECT COUNT(*) -- 같은 집계를 한 번 더 수행 (미사용)
FROM (SELECT R2.STD_NO FROM SUBJ_RESULT R2
WHERE R2.YY = :yy AND R2.SUBJ_DIV = 'P'
AND R2.STD_NO IN (SELECT S3.STD_NO FROM STD_MASTER S3
WHERE S3.GRDT_NO = (SELECT GRDT_NO FROM STD_MASTER
WHERE STD_NO = :std_no))
GROUP BY R2.STD_NO)) AS CNT
FROM (SELECT R.STD_NO
, NVL(SUM(DECODE(R.SHTM, '10', R.POINT)), 0) AS T1
, NVL(SUM(DECODE(R.SHTM, '20', R.POINT)), 0) AS T2
, NVL(SUM(DECODE(R.SHTM, '21', R.POINT)), 0) AS SUMMER
FROM SUBJ_RESULT R
WHERE R.YY = :yy AND R.SUBJ_DIV = 'P'
AND R.STD_NO IN (SELECT S2.STD_NO FROM STD_MASTER S2
WHERE S2.GRDT_NO = (SELECT GRDT_NO FROM STD_MASTER
WHERE STD_NO = :std_no))
GROUP BY R.STD_NO) C
WHERE C.STD_NO = :std_no) P,
-- 체력 점수: 행마다 환산표 스킴라 서브쿼리 실행
(SELECT SUM(X.FIT_POINT)/2 AS FIT_SCR
FROM (SELECT FR.SHTM
, (SELECT G.POINT FROM FIT_GRADE G
WHERE G.YY = FR.YY AND G.SHTM = FR.SHTM
AND NVL(FR.TOT_SCR, 0) BETWEEN G.SCR_FROM AND G.SCR_TO) AS FIT_POINT
, FN_CODE_NM('SHTM', FR.SHTM) AS SHTM_NM -- 미사용
FROM FITNESS_RESULT FR, STD_MASTER S
WHERE FR.STD_NO = S.STD_NO
AND FR.YY = :yy
AND S.STD_NO = :std_no) X) F,
-- 생활 점수
(SELECT (NVL(Y.T1, 0)*2 + NVL(Y.T2, 0)*2 + NVL(Y.SUMMER, 0))
/ (SELECT SUM(H.CREDIT) FROM SUBJ_RESULT H
WHERE H.YY = :yy AND H.SUBJ_DIV = 'L'
AND H.STD_NO = Y.STD_NO) AS LIFE_SCR
FROM (SELECT R.STD_NO
, MAX(DECODE(R.SHTM, '10', R.POINT)) AS T1
, MAX(DECODE(R.SHTM, '20', R.POINT)) AS T2
, MAX(DECODE(R.SHTM, '21', R.POINT)) AS SUMMER
FROM SUBJ_RESULT R, STD_MASTER S
WHERE R.STD_NO = S.STD_NO
AND R.YY = :yy AND R.SUBJ_DIV = 'L'
AND S.GRDT_NO = (SELECT GRDT_NO FROM STD_MASTER WHERE STD_NO = :std_no)
AND S.STD_NO = :std_no
GROUP BY R.STD_NO) Y) L,
-- 서열
(SELECT AVG(ALL_RANK) AS TOT_RANK
FROM TOTAL_RANK
WHERE STD_NO = :std_no
GROUP BY STD_NO) K;
```

개선 후: 대상 학생의 성적만 WITH절로 한 번 읽고, 영역별 점수는 조건부 집계(DECODE)로 계산했습니다. 환산표 조치는 스칼라 서브쿼리에서 조인으로 바꿨습니다. (sql/case1_after.sql)

```
-- 사례 1. 종합성적 조회 쿼리 - 개선 후 (가상, 일반화된 테이블명)
-- 대상 학생의 성적을 WITH절로 한 번만 읽고, 영역별 점수를 조건부 집계로 계산
WITH RSLT AS (
    -- 성적 테이블은 1회, 대상 학생만 조회
    SELECT R.SUBJ_DIV, R.SHTM, R.CREDIT, R.POINT
    FROM SUBJ_RESULT R
    WHERE R.YY = :yy
    AND R.STD_NO = :std_no
), STUDY AS (
    SELECT ROUND(SUM(CREDIT*POINT)/SUM(CREDIT), 2) AS STUDY_SCR
    FROM RSLT
    WHERE SUBJ_DIV IN (SELECT CD FROM COMMON_CODE
        WHERE GRP_CD = 'SUBJ_DIV' AND ATTR1 = 'G')
), PRAC AS (
    SELECT ROUND(NVL(SUM(DECODE(SHTM, '10', POINT)), 0)*0.2
        + NVL(SUM(DECODE(SHTM, '20', POINT)), 0)*0.2
        + NVL(SUM(DECODE(SHTM, '21', POINT)), 0)*0.6, 2) AS PRAC_SCR
    FROM RSLT
    WHERE SUBJ_DIV = 'P'
), LIFE AS (
    SELECT (NVL(MAX(DECODE(SHTM, '10', POINT)), 0)*2
        + NVL(MAX(DECODE(SHTM, '20', POINT)), 0)*2
        + NVL(MAX(DECODE(SHTM, '21', POINT)), 0)) / NULLIF(SUM(CREDIT), 0) AS LIFE_SCR
    FROM RSLT
    WHERE SUBJ_DIV = 'L'
), FIT AS (
    -- 스칼라 서브쿼리 > 조인
    SELECT SUM(G.POINT)/2 AS FIT_SCR
    FROM FITNESS_RESULT FR
    JOIN FIT_GRADE G
    ON G.YY = FR.YY AND G.SHTM = FR.SHTM
    AND NVL(FR.TOT_SCR, 0) BETWEEN G.SCR_FROM AND G.SCR_TO
    WHERE FR.YY = :yy
    AND FR.STD_NO = :std_no
), RNK AS (
    SELECT AVG(ALL_RANK) AS TOT_RANK
    FROM TOTAL_RANK
    WHERE STD_NO = :std_no
)
SELECT :yy AS YY
, T.STD_NO, T.STD_NM
, TO_CHAR(S.STUDY_SCR, 'FM990.00') AS STUDY_SCR
, TO_CHAR(P.PRAC_SCR, 'FM990.00') AS PRAC_SCR
, TO_CHAR(F.FIT_SCR, 'FM990.00') AS FIT_SCR
, TO_CHAR(L.LIFE_SCR, 'FM990.00') AS LIFE_SCR
, TO_CHAR(S.STUDY_SCR*0.5 + P.PRAC_SCR*0.25
    + F.FIT_SCR*0.15 + L.LIFE_SCR*0.1, 'FM990.00') AS TOT_SCR
, TO_CHAR(K.TOT_RANK, 'FM990.0') AS TOT_RANK
FROM STD_MASTER T
CROSS JOIN STUDY S
CROSS JOIN PRAC P
CROSS JOIN FIT F
CROSS JOIN LIFE L
CROSS JOIN RNK K
WHERE T.STD_NO = :std_no;
```

핵심은 학생 조건을 집계 이전으로 옮긴 것입니다. 기존에는 영역마다 기수 전체를 집계한 뒤 1명을 골랐지만, 개선 후에는 처음부터 대상 학생 1명의 데이터만 읽어 계산합니다. 이와 함께 반복되던 기수 조회 서브쿼리, 미사용 컬럼, 결과에 쓰이지 않던 COUNT 집계를 제거해 조회 시간을 약 10초에서 3초로 줄였습니다.

사례 2. 기능 추가에 따른 테이블 구조 변경 및 재구축

운영 데이터를 백업한 뒤 변경된 컬럼 구조로 테이블을 재생성해, 데이터 손실 없이 신규 기능을 반영했습니다.

문제 메뉴 기능 추가로 신규 컬럼 추가와 기존 컬럼 설정 변경이 필요했습니다. 운영 중인 데이터를 유지한 채 안전하게 구조를 바꿔야 했습니다.

조치

1. 작업 전 기존 테이블 전체를 백업 테이블로 복제
2. 변경된 컬럼 구조로 테이블 재생성
3. 백업 데이터를 신규 구조에 맞게 이관
4. 인덱스 변동 사항이 있는 경우 인덱스 재구성
5. 이관 전후 건수와 주요 데이터를 비교해 검증

결과 데이터 손실 없이 테이블 구조 변경을 완료하고 신규 기능이 정상 반영되었습니다.

예시 쿼리 (가상, 일반화된 테이블명)

신규 컬럼(MINOR_CD) 추가와 기존 컬럼(ADDR) 길이 변경을 가정한 절차입니다. (sql/case2_table_rebuild.sql)

```
-- 사례 2. 기능 추가에 따른 테이블 구조 변경 및 재구축 (가상, 일반화된 테이블명)
-- 신규 컬럼(MINOR_CD) 추가와 기존 컬럼(ADDR) 길이 변경을 가정한 절차

-- 1. 백업 테이블 생성 및 건수 확인
CREATE TABLE STUDENT_INFO_BAK_20230301 AS
SELECT * FROM STUDENT_INFO;

SELECT COUNT(*) FROM STUDENT_INFO;
SELECT COUNT(*) FROM STUDENT_INFO_BAK_20230301;

-- 2. 변경된 구조로 테이블 재생성
DROP TABLE STUDENT_INFO;

CREATE TABLE STUDENT_INFO (
    STD_NO    VARCHAR2(10) NOT NULL,
    STD_NM    VARCHAR2(50) NOT NULL,
    DEPT_CD   VARCHAR2(10),
    ADDR      VARCHAR2(100),      -- 50 > 100 변경
    MINOR_CD  VARCHAR2(10),      -- 신규 컬럼
    REG_DT    DATE DEFAULT SYSDATE,
    CONSTRAINT PK_STUDENT_INFO PRIMARY KEY (STD_NO)
);

-- 3. 데이터 이관
INSERT INTO STUDENT_INFO (STD_NO, STD_NM, DEPT_CD, ADDR, MINOR_CD, REG_DT)
SELECT STD_NO, STD_NM, DEPT_CD, ADDR, NULL, REG_DT
FROM STUDENT_INFO_BAK_20230301;
COMMIT;

-- 4. 인덱스 재구성 (변동 사항 반영)
CREATE INDEX IX_STUDENT_INFO_01 ON STUDENT_INFO (DEPT_CD, MINOR_CD);

-- 5. 검증: 건수 비교 + 데이터 차이 확인 (0건이면 정상)
SELECT COUNT(*) FROM STUDENT_INFO;

SELECT STD_NO, STD_NM, DEPT_CD, ADDR FROM STUDENT_INFO_BAK_20230301
MINUS
SELECT STD_NO, STD_NM, DEPT_CD, ADDR FROM STUDENT_INFO;
```

사례 3. 데이터 변경 작업 절차 수립 및 오삭제 복구

사전 백업과 담당자 이중 확인 절차를 지켰고, 조건 누락으로 데이터가 잘못 변경된 사고를 백업본과 Flashback Query로 복구했습니다.

작업 원칙 UPDATE/DELETE 등 데이터 변경 작업 시 다음 절차를 준수했습니다.

1. 변경 대상 데이터를 사전에 백업
2. 동일 조건으로 SELECT를 먼저 실행해 대상 건수 확인
3. 담당자와 이중 확인 후 실행
4. 작업 후 결과 건수 재검증

사고 상황 WHERE 조건 1개 요소 누락으로 변경 대상이 아닌 데이터까지 삭제 또는 수정되는 사고가 발생했습니다.

복구 조치 작업 전 확보한 백업본으로 원복하거나, Flashback Query(AS OF TIMESTAMP)로 변경 이전 시점의 데이터를 조회해 복구했습니다.

결과 및 개선 잘못 변경된 데이터를 변경 이전 상태로 복구했습니다. 이후 사전 SELECT 검증과 이중 확인 절차를 더 엄격히 적용했습니다.

예시 쿼리 (일반화된 테이블명)

특정 학생 1명의 성적을 수정하려다 조건 누락으로 전공코드는 다르면서 과목코드는 같은 다른 과목도 변경된 상황을 재구성했습니다.

(sql/case3_recovery.sql)

```
-- 사례 3. 데이터 변경 작업 절차 및 오변경 복구 (일반화된 테이블명)
-- 특정 학생 1명의 성적을 수정하려다 조건 누락으로
-- 전공코드는 다르면서 과목코드는 같은 다른 과목도 변경된 상황을 재구성

-- 1. 사전 백업
CREATE TABLE COURSE_GRADE_BAK_20230615 AS
SELECT * FROM COURSE_GRADE;

-- 2. 사전 SELECT로 대상 건수 확인
SELECT COUNT(*) FROM COURSE_GRADE
WHERE YY = '2023' AND TERM = '1'
AND STD_NO = '20334' AND SUBJ_CD = '10553'; -- 23년 1학기 -- 20334번 학생의 10553과목

-- 3. 사고: DEPT_CD 조건 누락
-- 이후 전공코드가 다른데 과목코드는 같은 또 다른 과목도 변경 처리된 것을 파악
```

```

UPDATE COURSE_GRADE
SET GRADE = 'B+', POINT = 3.5
WHERE YY = '2023' AND TERM = '1'
AND STD_NO = '20334' AND SUBJ_CD = '10553';
-- AND DEPT_CD = '0029' 누락

-- 복구 방법 A: 사전 백업본으로 원복
UPDATE COURSE_GRADE G
SET (GRADE, POINT) = (SELECT B.GRADE, B.POINT
FROM COURSE_GRADE_BAK_20230615 B
WHERE B.STD_NO = G.STD_NO
AND B.SUBJ_CD = G.SUBJ_CD
AND B.YY = G.YY
AND B.TERM = G.TERM
AND B.DEPT_CD = G.DEPT_CD)
WHERE G.YY = '2023' AND G.TERM = '1'
AND G.STD_NO = '20334' AND SUBJ_CD = '10553';
COMMIT;

-- 복구 방법 B: Flashback Query로 변경 이전 시점 데이터 조회 후 복구
-- 10분 전 시점의 데이터 확인
SELECT STD_NO, SUBJ_CD, GRADE, POINT
FROM COURSE_GRADE AS OF TIMESTAMP (SYSTIMESTAMP - INTERVAL '10' MINUTE)
WHERE YY = '2023' AND TERM = '1'
AND STD_NO = '20334' AND SUBJ_CD = '10553';

-- 해당 시점 데이터로 원복
UPDATE COURSE_GRADE G
SET (GRADE, POINT) = (SELECT F.GRADE, F.POINT
FROM COURSE_GRADE AS OF TIMESTAMP
(SYSTIMESTAMP - INTERVAL '10' MINUTE) F
WHERE F.STD_NO = G.STD_NO
AND F.SUBJ_CD = G.SUBJ_CD
AND F.YY = G.YY
AND F.TERM = G.TERM
AND F.DEPT_CD = G.DEPT_CD)
WHERE G.YY = '2023' AND G.TERM = '1'
AND G.STD_NO = '20334' AND G.SUBJ_CD = '10553';
COMMIT;

-- 원복 후 원래 의도했던 1건 수정을 올바른 조건(DEPT_CD 포함)으로 다시 수행

```

원복 후에는 원래 의도했던 1건 수정을 올바른 조건으로 다시 수행했습니다. Flashback은 Undo 보존 기간 안에서만 가능하므로 사전 백업을 기본 원칙으로 두었습니다.

사례 4. 데이터 장애 원인 추적 및 정정

동명이인 두 학생의 신상정보가 서로 바뀌어 증명서에 잘못 출력되는 문제를, 관련 테이블을 전수 추적해 정정했습니다.

문제 외부 인터넷 증명서 발급 시 동명이인 두 학생(교번 18434, 18435)의 신상정보가 서로 바뀌어 출력된다는 민원이 접수되었습니다. 신상정보는 DB 담당자 외에는 수정할 수 없어 DB 작업으로 정정이 필요했습니다.

분석 증명서가 참조하는 뷰와 원본 테이블을 확인한 뒤, 신상정보 컬럼을 가진 테이블을 디셔너리 뷰로 전수 조회해 수정 대상 2개 테이블을 특정했습니다. 원인은 최초 데이터 입력 시 행정실 실무자의 입력 실수였습니다.

조치

1. 두 학생의 데이터를 행정실 원본 서류와 대조해 오입력 확인
2. 수정 대상 테이블의 두 학생 데이터 백업
3. 백업본을 기준으로 두 교번의 신상정보를 맞교환
4. 백업본과 비교해 정확히 교환되었는지 검증 후 행정실 담당자와 이중 확인

결과 2개 테이블의 신상정보를 정정하고 증명서 재출력으로 정상 출력을 확인해 민원을 해결했습니다.

예시 쿼리 (가상, 일반화된 명칭)

증명서 > 참조 테이블 > 신상정보 컬럼 보유 테이블 순으로 추적한 뒤, 백업본을 기준으로 두 교번의 데이터를 맞교환하고 검증했습니다.
([sql/case4_trace.sql](#))

```

-- 사례 4. 동명이인 신상정보 오입력 추적 및 정정 (가상, 일반화된 명칭)
-- 상황: 동명이인인 교번 18434, 18435 두 학생의 신상정보가 서로 바뀌어 증명서에 출력됨
-- 원인: 최초 데이터 입력 시 행정실 실무자의 입력 실수
-- (신상정보는 DB 담당자 외 수정 불가하여 DB 작업으로 정정)

-- 1. 증명서가 참조하는 원본 테이블 확인
SELECT REFERENCED_NAME, REFERENCED_TYPE
FROM ALL_DEPENDENCIES

```

```

WHERE NAME = 'CERT_STD_INFO_V';          -- 증명서용 신상정보 뷰

-- 2. 신상정보 컬럼을 가진 테이블 전수 확인 (증명서 외 화면에도 쓰이는 테이블까지)
SELECT TABLE_NAME, COLUMN_NAME
FROM ALL_TAB_COLUMNS
WHERE TABLE_NAME IN (SELECT TABLE_NAME FROM ALL_TAB_COLUMNS
                        WHERE COLUMN_NAME = 'STD_NO')
AND COLUMN_NAME IN ('BIRTH_DT', 'ADDR', 'TEL_NO', 'EMAIL')
ORDER BY TABLE_NAME, COLUMN_NAME;
-- > 수정 대상: STD_MASTER(학적 기본), STD_PERSONAL(신상)

-- 3. 두 학생 데이터 확인 (행정실 원본 서류와 대조)
SELECT STD_NO, STD_NM, BIRTH_DT FROM STD_MASTER
WHERE STD_NO IN ('18434', '18435');
SELECT STD_NO, ADDR, TEL_NO, EMAIL FROM STD_PERSONAL
WHERE STD_NO IN ('18434', '18435');

-- 4. 수정 전 백업 (두 학생 데이터만)
CREATE TABLE STD_MASTER_BAK_20230410 AS
SELECT * FROM STD_MASTER WHERE STD_NO IN ('18434', '18435');
CREATE TABLE STD_PERSONAL_BAK_20230410 AS
SELECT * FROM STD_PERSONAL WHERE STD_NO IN ('18434', '18435');

-- 5. 두 교번의 신상정보 맞교환 (백업본에서 상대 교번의 값을 가져옴)
UPDATE STD_MASTER M
SET M.BIRTH_DT = (SELECT B.BIRTH_DT
                  FROM STD_MASTER_BAK_20230410 B
                  WHERE B.STD_NO = DECODE(M.STD_NO, '18434', '18435', '18435', '18434'))
WHERE M.STD_NO IN ('18434', '18435');

UPDATE STD_PERSONAL P
SET (P.ADDR, P.TEL_NO, P.EMAIL) =
(SELECT B.ADDR, B.TEL_NO, B.EMAIL
 FROM STD_PERSONAL_BAK_20230410 B
 WHERE B.STD_NO = DECODE(P.STD_NO, '18434', '18435', '18435', '18434'))
WHERE P.STD_NO IN ('18434', '18435');

-- 6. 검증: 백업본의 교번을 바꿔 비교 (모두 0건이면 정상 교환)
SELECT DECODE(STD_NO, '18434', '18435', '18435', '18434') AS STD_NO, BIRTH_DT
FROM STD_MASTER_BAK_20230410
MINUS
SELECT STD_NO, BIRTH_DT FROM STD_MASTER WHERE STD_NO IN ('18434', '18435');

SELECT DECODE(STD_NO, '18434', '18435', '18435', '18434') AS STD_NO, ADDR, TEL_NO, EMAIL
FROM STD_PERSONAL_BAK_20230410
MINUS
SELECT STD_NO, ADDR, TEL_NO, EMAIL FROM STD_PERSONAL WHERE STD_NO IN ('18434', '18435');

-- 7. 행정실 담당자와 이중 확인 후 반영, 증명서 재출력으로 최종 확인
COMMIT;

```

두 행을 서로 바꿀 때 한쪽을 먼저 덮어쓰면 원래 값이 사라지기 때문에, 백업본에서 DECODE로 상대 교번의 값을 가져오는 방식으로 교환했습니다. 검증 단계에서도 백업본의 교번을 뒤집어 현재 데이터와 비교해, 0건이 나오는 것으로 정확한 교환을 확인했습니다.